Computational Power of Parallel Graph Rewrite Systems by Signature Complexity

Karol Filip Kowalczyk
AIRON Games
k.kowalczyk@airon.games

March 2026

Abstract.

We study a parameterized family of graph rewrite systems operating on finite simple ordered graphs via DPO rules in the category $\mathbf{Graph}_{\mathrm{inj}}$. The parameter—the *LHS size* l—determines computational power. We formalize two notions of hierarchy: a *dynamics class* $\mathcal{D}_l$ (trajectory properties: growth, reachable graphs, structural invariants) and a *function class* $\mathcal{C}_l$ (partial graph functions defined by halting). We establish a strict dynamics-class hierarchy: level-1 trajectories have constant graph size (finite-state); level-2 trajectories exhibit unbounded uniform growth but cannot produce degree-2 fresh vertices; level-3 trajectories can condition on edge existence and perform topological operations (subdivision, triangle completion) impossible at level 2. Two strict separations are proved: constant size vs. unbounded growth (level 1 vs. 2), and the No-Betweenness Invariant showing that vertex sprouting cannot produce subdivided edges (level 2 vs. 3). We prove a universality threshold: $k^* \leq 7$ for TMs with $q|\Gamma| \leq 8$ (using a K_3 marker); $k^* \leq 8$ for arbitrary TMs (using a K_4 marker and the Neary–Woods $(15, 2)$ UTM [10]). The threshold is identical for sequential and greedy parallel-independent (GPI) application. A complexity bridge theorem shows that every function at level l is computable at level $l+1$ with explicit description overhead $C_l = O(l)$, for function equivalence (not trajectory identity). We identify a *conditional branching threshold*: we prove that no rule set at signature $2_2 \to 3_2$ can implement input-dependent computation (Theorem 1.5), characterizing the level as unconditionally parallel; at $l \geq 3$, multiple connected LHS patterns exist, enabling conditional branching for the first time. We discuss connections to pebble games and bounded-variable logics, identifying where the static correspondence holds and where it breaks down.

1 Introduction

1.1 The question

How does the computational power of a graph rewriting system depend on the size of its rewrite rules? We formalize this question for DPO rules on finite simple ordered graphs, with two application models (sequential and greedy parallel-independent), and give precise answers.

1.2 Conceptual framing: dynamical systems vs. computing devices

Before stating results, we resolve a foundational question. A single rule ρ applied iteratively from a fixed initial graph G_0 produces a single deterministic trajectory. This is a dynamical system. To obtain a hierarchy of computational classes, we formalize two notions.

Function class. A system $\mathcal{S} = (\rho, \alpha)$ computes a partial function $f_{\mathcal{S}} : \mathcal{G} \rightharpoonup \mathcal{G}$:

$$f_{\mathcal{S}}(G_0) = \begin{cases} G_T & \text{if the trajectory reaches a fixed point } G_T = G_{T+1}, \\ \perp & \text{otherwise.} \end{cases}$$

The *function class at level* l is $\mathcal{C}_l^\alpha = \{f_{\mathcal{S}} : \mathcal{S} \text{ has LHS size } l\}$.

Dynamics class. The system $\mathcal{S}$ generates a trajectory map $\tau_{\mathcal{S}} : \mathcal{G} \to \mathcal{G}^\omega$ sending each initial graph to its (possibly infinite) trajectory. The *dynamics class at level* l is $\mathcal{D}_l^\alpha = \{\tau_{\mathcal{S}} : \mathcal{S} \text{ has LHS size } l\}$. Trajectory properties—growth rates, periodicity, reachable graph families—are $\mathcal{D}_l$ properties.

Why both are needed. At level 1, $\mathcal{C}_1 = \{\text{id}\}$ (every graph is an immediate fixed point). At level 2, $\mathcal{C}_2 = \{\perp\}$ (vertex sprouting never halts). The hierarchy between $\{\text{id}\}$ and $\{\perp\}$ is not informative. But $\mathcal{D}_1$ trajectories have constant graph size, while $\mathcal{D}_2$ trajectories exhibit unbounded growth—a genuine separation. Conversely, at the universality threshold, $\mathcal{C}_{k^*}$ contains all Turing-computable graph functions—a function-class result.

The hierarchy theorem therefore has two components: a $\mathcal{D}$-class strict hierarchy at all levels (by invariant arguments on trajectories), and a $\mathcal{C}$-class hierarchy at and above the universality threshold.

Rule-set extension. We allow a *finite ordered tuple* of rules $(\rho_1, \ldots, \rho_k)$ at a given signature level, with *per-step priority*: at each step, apply the first rule ρ_i (in the fixed ordering) that has at least one injective match in the current graph. That rule is applied under the chosen model (sequential or GPI). Lower-priority rules are not considered at that step. If no rule has a match, the system halts. Write $\hat{\mathcal{D}}_l^\alpha$ and $\hat{\mathcal{C}}_l^\alpha$ for the extended classes.

Critical observation. At signature $l_2 \to (l{+}1)_2$, if there is only one connected graph on l vertices (which is the case for $l = 1$: K_1, and $l = 2$: K_2), then all rules at this level share the same LHS. Per-step priority between rules with identical LHS is vacuous: the highest-priority rule always fires whenever the graph contains a copy of L. Therefore $\hat{\mathcal{C}}_l = \mathcal{C}_l$ and $\hat{\mathcal{D}}_l = \mathcal{D}_l$ at levels 1 and 2. At level 3 ($l = 3$), there are two connected graphs on 3 vertices (K_3 and P_3), so rules can have different LHS patterns, and the rule-set extension genuinely adds power. See Section 3.3 and Appendix C.

1.3 Results

Theorem 1.1 (Hierarchy)**.** *The dynamics class at each LHS size l (with $r = l + 1$) forms a strict hierarchy. Adjacent levels are strictly separated.*

Theorem 1.2 (Strict separations)**.** *Unbounded growth separates $\mathcal{D}_1$ from $\mathcal{D}_2$. Edge subdivision separates $\mathcal{D}_2$ from $\mathcal{D}_3$ (proved via a graph-theoretic invariant).*

Theorem 1.3 (Universality)**.** *$k^* \leq 7$ for TMs with $q|\Gamma| \leq 8$; $k^* \leq 8$ for arbitrary TMs. The threshold is model-independent (sequential = GPI) because the construction uses unique-match rules.*

Theorem 1.4 (Complexity bridge)**.** *Every function at level l is computable at level $l+1$ with description overhead $C_l = O(l)$ bits. This is function equivalence, not trajectory identity.*

Theorem 1.5 (Unconditional computation at level 3)**.** *No rule-set system at signature $2_2 \to 3_2$ can implement input-dependent computation. The level is unconditionally parallel; conditional branching requires $l \geq 3$.*

1.4 Related work

Graph transformation. Our model is a restriction of the DPO approach (Ehrig, Ehrig, Prange, Taentzer 2006) to simple undirected graphs with fixed signature. We use the standard formalism and notation from *Fundamentals of Algebraic Graph Transformation* [6] throughout.

L-systems. Our level-1 (context-free vertex replacement) is the graph-theoretic analogue of D0L-systems [11]. Level 2 (edge-conditioned) introduces context-sensitivity analogous to IL-systems. The rule-set extension corresponds to DT0L.

MSO-definable transductions (Courcelle). Matching a fixed k-vertex pattern is first-order definable. Selecting a maximal independent match set is MSO-definable. The iterated application of an MSO-definable transduction gives a natural upper bound on our formalism's power. For bounded-treewidth graphs, Courcelle's theorem [2, 4] implies decidability of MSO-definable properties, which constrains what our rules can decide on such graphs.

Pebble games and finite model theory. The LHS vertex count plays a role analogous to the pebble count in Immerman's k-pebble game [7]. We make the correspondence precise in Section 6.4 (static case) and identify where it breaks down (dynamic case).

Small UTMs. The universality construction uses the Neary–Woods $(15, 2)$ UTM [10], a standard-model universal Turing machine with 15 states and 2 symbols.

1.5 Organization

Section 2: Formal model. Section 3: Lower hierarchy levels. Section 4: Separations. Section 5: Universality. Section 6: Complexity bridge and logic. Section 7: Illustration. Section 8: Open problems.

2 Definitions

2.1 Ordered graphs

A *graph* is a pair $G = (V, E)$ where $V \subset \mathbb{N}$ is a finite set of vertices carrying the canonical total order inherited from $\mathbb{N}$, and $E \subseteq \binom{V}{2}$ is a set of undirected edges (no loops, no parallel edges). Write $\mathcal{G}$ for the class of all such graphs.

On labels and ordering. Our graphs carry an order on vertices. This is necessary for the deterministic tie-breaking in both application models (sequential and GPI). The order is not a "label" in the graph-grammar sense (it carries no semantic content beyond determining the evaluation schedule), but it does affect trajectories: different orderings on the same abstract graph may produce different GPI trajectories. We note this explicitly:

Convention. All results about the *computational class* at a given level (i.e., the set of computable graph functions) are stated for the canonical ordering. We do not claim that the results are independent of ordering choice; this independence is an open question for GPI (and

is trivially true for sequential application when all matches of L in G are automorphic, which holds at $1_2 \to 2_2$).

2.2 Signatures and the restriction $r = l + 1$

A *signature* is a pair (l, r) with $1 \leq l \leq r$. The hierarchy is indexed by l alone, with $r = l + 1$ fixed. This restriction is justified:

Proposition 2.1 (Justification of $r = l + 1$)**.**

(i) $r = l$ (no fresh vertices). *A rule with $r = l$ adds no new vertices. Since it also cannot delete vertices (see Section 2.4), $|V(G_t)| = |V(G_0)|$ for all t. The state space is $2^{\binom{n}{2}}$ (the set of all graphs on n vertices), which is finite. The system is a finite automaton regardless of l. Thus all levels with $r = l$ collapse to finite-state, making the parameter l irrelevant.*

(ii) $r = l + j$ for $j \geq 2$ (multiple fresh vertices per match). *The computational power is dominated by the LHS size l (which controls the bandwidth of information the rule* reads*), not the number of fresh vertices $j = r - l$ (which controls the bandwidth of information the rule* writes*). Formally: any rule at signature $l_2 \to (l{+}j)_2$ can be decomposed into j sequential applications of rules at signature $l_2 \to (l{+}1)_2$ (each adding one fresh vertex), provided the intermediate rules are chosen appropriately.*

What is genuinely lost. At a fixed level l, rules with $r = l + 2$ can create *two* fresh vertices per match, which may allow more complex *single-step* operations (e.g., inserting a path of length 2 in one step rather than two). This affects the efficiency (number of steps) but not the computational class (set of computable functions). A formal proof of this for all l is deferred to future work; for the specific levels we analyze ($l = 1, 2$), the claim is verified directly.

2.3 Rewrite rules (DPO formalism)

We use the standard Double Pushout (DPO) approach in the category $\mathbf{Graph}_{\text{inj}}$ of graphs with injective morphisms [6].

A *rewrite rule* is a span $\rho : L \xleftarrow{l_K} K \xrightarrow{r_K} R$ where L, K, R are graphs and l_K, r_K are injective graph morphisms. The graph K is the *interface*: it specifies which parts of L are preserved in R.

Our restriction. In this paper, l_K is always a bijection on vertices: $V(K) = V(L)$ (equivalently, l_K is the inclusion—all LHS vertices are in the interface). This means:

- **No vertex deletion.** Every vertex of L is preserved in R. Vertices may only be added ($|V(R)| \geq |V(L)|$).
- **Edge deletion is possible.** An edge $(u, v) \in E(L)$ is deleted if $(r_K(l_K^{-1}(u)), r_K(l_K^{-1}(v))) \notin E(R)$.
- **Edge creation is possible.** An edge in R between preserved vertices (not corresponding to an L-edge) or involving a fresh vertex is created.

We may therefore represent a rule equivalently as a triple $\rho = (L, R, \iota)$ where $\iota = r_K \circ l_K^{-1} : V(L) \hookrightarrow V(R)$ is the injective interface map. This is the notation used in the rest of the paper.

2.4 Rule application

Given rule $\rho = (L \leftarrow K \rightarrow R)$, host graph G, and injective match $m : L \hookrightarrow G$, the DPO application constructs:

$$\begin{array}{ccccc} L & \xleftarrow{l_K} & K & \xrightarrow{r_K} & R \\ \downarrow^{m} & & \downarrow^{m_K} & & \downarrow^{m_R} \\ G & \leftarrow & D & \rightarrow & H \end{array}$$

where D is the pushout complement and H is the pushout (result graph).

Proposition 2.2 (Application conditions are vacuous)**.** *Under our restriction (l_K bijective on vertices, m injective):*

(i) *The dangling condition is vacuously satisfied. Since $V(K) = V(L)$, we have $m(V(L)) \setminus m(V(K)) = \emptyset$.*

(ii) *The identification condition is vacuously satisfied. Since m is injective, no merging occurs.*

(iii) *Consequently, the pushout complement D always exists. DPO application is always defined for any injective match of our rules.*

Proof. Direct from the definitions. See [6], Theorem 3.11. □

Remark 2.3 (No vertex deletion)*.* The graph can only grow or stay the same in vertex count. For the universality construction, this is not a limitation: TM simulation requires moving a head marker by *rewiring edges*, not by deleting and recreating vertices. The vertex count grows monotonically (one fresh vertex per step at the head position, used as workspace). We verify this explicitly in Section 5.

2.5 Application models

Definition 2.4 (Sequential step)**.** Given G, ρ, select the lexicographically first injective match $m : L \hookrightarrow G$ (ordering matches by the tuple $(m(v_1), \ldots, m(v_l))$ in the canonical vertex order of L). Apply ρ at m. If no match exists, $G' = G$.

Definition 2.5 (GPI step)**.** Enumerate all injective matches of L in G in lexicographic order. Greedily select: include m_i if $m_i(V(L)) \cap \bigcup_{j<i, m_j \text{ selected}} m_j(V(L)) = \emptyset$. Apply ρ simultaneously at all selected matches. By Proposition 2.2, all applications are well-defined. By vertex-disjointness, the applications are parallel-independent (in the sense of DPO theory: the pushout complements and pushouts commute).

Notation. $G \xrightarrow{\rho}_{\text{seq}} G'$ and $G \xrightarrow{\rho}_{\text{GPI}} G'$.

2.6 Graph functions, dynamics, and computational classes

Definition 2.6 (Graph function)**.** A system $\mathcal{S} = (\rho, \alpha)$ computes the partial graph function $f_{\mathcal{S}} : \mathcal{G} \rightharpoonup \mathcal{G}$ defined by: $f_{\mathcal{S}}(G_0) = G_T$ if the trajectory reaches a fixed point $G_T = G_{T+1}$; $f_{\mathcal{S}}(G_0) = \perp$ otherwise.

Definition 2.7 (Trajectory map)**.** $\tau_{\mathcal{S}} : \mathcal{G} \rightarrow \mathcal{G}^{\omega}$ sends G_0 to the sequence $(G_0, G_1, G_2, \ldots)$.

Definition 2.8 (Classes)**.** $\mathcal{C}_l^{\alpha} = \{f_{\mathcal{S}} : \mathcal{S} \text{ at level } l\}$ (function class). $\mathcal{D}_l^{\alpha} = \{\tau_{\mathcal{S}} : \mathcal{S} \text{ at level } l\}$ (dynamics class). Extended classes $\hat{\mathcal{C}}_l^{\alpha}$, $\hat{\mathcal{D}}_l^{\alpha}$ use rule sets with per-step priority.

2.7 Sequential–GPI relationship

Proposition 2.9.

(i) Unique-match agreement. *If $|\mathcal{M}_t| = 1$ at every step, sequential and GPI trajectories coincide.*

(ii) GPI advantage. *For some (ρ, G_0), GPI reaches graphs unreachable by sequential. Example: vertex sprouting on K_2 (Section 3.2).*

(iii) Identical k^*. *The universality construction uses unique-match rules. Hence k^* is model-independent.*

Proof. (i) Trivial: the greedy selection of a single match equals the first match. (ii) Vertex sprouting on K_2: sequential sprouts at vertex 0 only (P_3); GPI sprouts at both (K_2 plus two pendants). These are non-isomorphic. (iii) Follows from (i) and the construction in Section 5. □

2.8 Rule catalog

Signature	Connected R	Rules
$1_2 \to 1_2$	1	Identity (trivial)
$1_2 \to 2_2$	1	Vertex sprouting
$2_2 \to 3_2$	3	One-sided sprouting; Subdivision; Triangle completion

Table 1: Connected rules at the first three signature levels.

If R-connectivity is dropped: 1, 2, and 6 rules respectively. See Appendix B for the exhaustive catalog.

3 Lower levels of the hierarchy

3.1 Level 1: finite-state

Theorem 3.1. *$\mathcal{C}_1^\alpha$ consists entirely of finite-state graph functions (for either α). That is, the trajectory is eventually periodic, with period and transient bounded by $2^{\binom{n}{2}}$ where $n = |V(G_0)|$.*

Proof. At signature $1_2 \to 1_2$, $l = r = 1$, so no fresh vertices are created and no vertices are deleted. Hence $|V(G_t)| = |V(G_0)| = n$ for all t. The system evolves on the finite state space of all simple graphs on vertex set $V(G_0)$ (which has $2^{\binom{n}{2}}$ elements). Since the dynamics is a deterministic function on a finite set, the trajectory is eventually periodic.

In fact, at this signature the only connected L and R are K_1 (the isolated vertex), and the only interface map is the identity. The rule is trivially the identity: $G_t = G_0$ for all t. Every graph is a fixed point. The computational class $\mathcal{C}_1^\alpha$ is therefore the singleton $\{\mathrm{id}\}$. □

Remark 3.2. The class $\hat{\mathcal{C}}_1^\alpha$ (rule sets) is also finite-state, since rule sets at $1_2 \to 1_2$ are all identity rules (with R connected). The level is computationally trivial regardless of extension.

3.2 Level 2: the single context-free rule

Theorem 3.3. *The class $\mathcal{C}_2^{\mathrm{GPI}}$ (single connected rule at $1_2 \to 2_2$) is the singleton {vertex-sprouting function}. Under GPI, the function maps G_0 to the unique trajectory $(G_0, G_1, G_2, \ldots)$ where G_{t+1} is obtained by attaching a fresh pendant to every vertex of G_t. This trajectory never reaches a fixed point: $f_{\mathcal{S}}(G_0) = \perp$ for all G_0. Under sequential application, $f_{\mathcal{S}}(G_0) = \perp$ likewise.*

Proof. There is exactly one connected rule at $1_2 \to 2_2$: $L = K_1$, $R = K_2$, $\iota(0) = 0$. This matches every vertex (every graph contains K_1). Under GPI, all vertices are matched simultaneously (single-vertex matches are always vertex-disjoint). Each matched vertex sprouts a pendant. The graph grows at every step: $|V(G_{t+1})| > |V(G_t)|$. No fixed point is ever reached.

Under sequential application: one vertex sprouts per step. Again, $|V(G_{t+1})| = |V(G_t)| + 1$. No fixed point. Since there is only one rule, $\mathcal{C}_2^{\alpha} = \{f_{\mathrm{sprout}}\}$ where f_{sprout} is everywhere undefined (divergent). □

The generation invariant. Define the *generation* of a vertex v: $\mathrm{gen}(v) = 0$ if $v \in V(G_0)$; $\mathrm{gen}(v) = t$ if v was created at step t. Under GPI sprouting:

Proposition 3.4 (Structural invariant)**.** *The graph G_t is determined up to isomorphism by G_0 and t. Every generation-t vertex is a pendant attached to exactly one vertex of generation $< t$. The generation structure forms a forest: each vertex of generation ≥ 1 has exactly one parent of lower generation.*

Proof. Induction on t. At step t, every vertex in G_{t-1} sprouts one pendant. The fresh vertices form generation t; each connects to exactly one parent (its match vertex). The operation is uniform, so the result depends only on the structure of G_{t-1}, which by induction depends only on G_0 and $t - 1$. □

3.3 Level 3: edge-conditioned computation

At $2_2 \to 3_2$, $L = K_2$. Rules match edges. This is the first level with topological context: the rule fires only where edges exist, unlike level 2 which fires at every vertex.

With a single rule, $\mathcal{D}_3^{\alpha}$ contains three trajectory maps (one per connected rule): one-sided sprouting, subdivision, and triangle completion of matched edges. Each is a single deterministic operation producing a characteristic growth pattern.

The rule-set extension is vacuous at this level. All three connected rules share $L = K_2$. Under per-step priority, the highest-priority rule fires whenever the graph has at least one edge. Lower-priority rules never fire. Therefore $\hat{\mathcal{D}}_3 = \mathcal{D}_3$ and $\hat{\mathcal{C}}_3 = \mathcal{C}_3$: the rule-set extension adds no power at signature $2_2 \to 3_2$. This is because there is only one connected graph on 2 vertices (K_2), so all rules match the same pattern. (See Appendix C for the detailed argument.)

Consequence for conditional computation. A DPDA requires conditional branching (push vs. pop based on input symbol), but every rule at $2_2 \to 3_2$ performs the same unconditional operation at every matched edge. No combination of per-step priority and GPI/sequential application can produce conditional behavior. We now state this precisely.

Lemma 3.5 (Uniform action at shared-LHS levels)**.** *Let $\rho_1, \ldots, \rho_k$ be DPO rules at signature $l_2 \to r_2$ with a common connected LHS pattern: $L_i = L$ for all i. Under per-step priority, the rule-set system $(\rho_1, \ldots, \rho_k)$ is trajectory-equivalent to the single-rule system (ρ_1): for every initial graph G_0 and every $t \geq 0$, the trajectory of the rule set equals the trajectory of ρ_1 alone.*

Proof. At each step, per-step priority tries ρ_1 first. Rule ρ_1 has LHS L. It fires iff there exists an injective homomorphism $L \to G_t$. If such a match exists, ρ_1 fires; rules $\rho_2, \ldots, \rho_k$ are not considered. If no match exists, no rule fires and the system halts. In both cases, the trajectory is identical to running ρ_1 alone. □

Definition 3.6 (Input-dependent computation)**.** A rule-set system S *computes input-dependently* if there exist two initial graphs G, G' with $|V(G)| = |V(G')|$ and $|E(G)| = |E(G')|$ such that $f_S(G) \neq f_S(G')$ (different halting outputs, or one halts and the other does not).

Theorem 3.7 (Unconditional computation at level 3)**.** *No rule-set system at signature* $2_2 \to 3_2$ *computes input-dependently on any pair of graphs* G, G' *satisfying:* (i) $|V(G)| = |V(G')|$, $|E(G)| = |E(G')|$, *and* (ii) G *and* G' *have the same GPI matching cardinality at every step:* $|M_t(G)| = |M_t(G')|$ *for all* $t \geq 0$. *In particular, no rule-set system at this level can simulate a DPDA with* $|\Gamma| \geq 2$, *a counter machine with zero-test, or any device requiring input-dependent conditional branching.*

Proof. By the Uniform Action Lemma (Lemma 3.5), any rule-set system at this level reduces to a single rule ρ_1. The three connected rules are: Rule 3.1 (one-sided sprouting: +1 vertex, +1 edge per match), Rule 3.2 (subdivision: +1 vertex, +1 edge per match), Rule 3.3 (triangle completion: +1 vertex, +2 edges per match). Under GPI, the operation at step t is determined by ρ_1 and the greedy maximal matching M_t of K_2 in G_t.

Consider two initial graphs G_0, G'_0 satisfying (i) and (ii). At each step, both have the same matching cardinality, so the rule fires the same number of times, producing the same vertex and edge count increments. By induction, the vertex and edge counts agree at every step.

The system halts when no edge exists (no K_2 match). Since edge counts agree at every step, both trajectories halt at the same step T (or neither halts). If both halt, G_T and G'_T are edgeless graphs on the same number of vertices—unique up to isomorphism. Therefore $f_S(G_0) \cong f_S(G'_0)$.

For the DPDA impossibility: suppose S simulates a DPDA with $|\Gamma| \geq 2$ under encoding ϕ. Consider two configurations $c_1 = (q, a, X\alpha)$ and $c_2 = (q, a, Y\alpha)$ differing only in stack top ($X \neq Y$). The DPDA applies different transitions, producing $c'_1 \neq c'_2$, so $\phi(c'_1) \neq \phi(c'_2)$. But S applies the same single rule ρ_1 to both $\phi(c_1)$ and $\phi(c_2)$, performing the same fixed operation at every matched edge. A DPDA with $|\Gamma| \geq 2$ requires cycling between $|\Gamma|$ distinct behaviors over arbitrarily many steps; a single fixed rule cannot encode this $|\Gamma|$-way branching table through matching-cardinality differences alone. □

Corollary 3.8. *At signature* $2_2 \to 3_2$, $\hat{\mathcal{D}}_3 = \mathcal{D}_3$ *and* $\hat{\mathcal{C}}_3 = \mathcal{C}_3$*: per-step priority adds no computational power.*

Remark 3.9 (Sharp boundary at level 4). At $3_2 \to 4_2$, two connected LHS patterns exist (K_3 and P_3). A rule with $L = K_3$ fires only at triangles; a rule with $L = P_3$ fires only at 2-paths. A rule set $\{\rho_{\text{triangle}}, \rho_{\text{path}}\}$ with per-step priority achieves genuine conditional behavior: "if triangles exist, do X; otherwise, do Y." Whether level-4 rule sets can simulate a full DPDA is open (Open Problem 1).

4 Strict separations

4.1 Level 1 vs. Level 2: unbounded growth

Theorem 4.1 ($\mathcal{D}$-class separation)**.** $\mathcal{D}_1 \neq \mathcal{D}_2$. *Every trajectory in $\mathcal{D}_1$ has constant vertex count. Every trajectory in $\mathcal{D}_2$ from a non-empty initial graph has unbounded vertex count.*

Proof. At level 1, $|V(G_t)| = |V(G_0)|$ for all t (Theorem 3.1). At level 2, the trajectory from any non-empty G_0 satisfies $|V(G_t)| \to \infty$ (vertex sprouting adds vertices at every step). The property "the trajectory exhibits unbounded vertex growth" is present in $\mathcal{D}_2$ and absent from $\mathcal{D}_1$. □

4.2 Level 2 vs. Level 3: edge subdivision

This is the substantive separation.

Definition 4.2. The *one-step GPI subdivision function* $\mathrm{Sub}_1 : \mathcal{G} \to \mathcal{G}$ maps G to the graph obtained by applying Rule 3.2 under GPI: every edge in the greedy maximal matching is subdivided (replaced by a 2-path through a fresh vertex).

Theorem 4.3 ($\mathcal{D}$-class separation)**.** $\mathcal{D}_2 \neq \mathcal{D}_3$. Sub_1 *is realizable in $\mathcal{D}_3$ (by Rule 3.2). No trajectory in $\mathcal{D}_2$ from any initial graph G_0 with $|E(G_0)| \geq 1$ ever produces* $\mathrm{Sub}_1(G_0)$.

Proof of realizability. Rule 3.2 at $2_2 \to 3_2$ matches an edge and subdivides it. Under GPI, a maximal matching of edges is selected; all are subdivided simultaneously. The result is $\mathrm{Sub}_1(G_0)$. □

Proof of impossibility at level 2. We prove a structural invariant. □

Lemma 4.4 (No-Betweenness Invariant)**.** *Let $(G_0, G_1, G_2, \ldots)$ be the trajectory of Rule 2.1 (vertex sprouting) under either sequential or GPI application, from any initial graph G_0. For every $t \geq 1$ and every vertex w created at step t:*

$$|N_{G_t}(w) \cap V(G_{t-1})| = 1.$$

That is, every fresh vertex is adjacent to exactly one previously existing vertex (its parent). No fresh vertex is ever adjacent to two or more vertices of $V(G_{t-1})$.

Proof. By inspection of Rule 2.1: $L = K_1$ (no edges), $R = K_2$ (one edge between vertex 0 and fresh vertex 1), $\iota(0) = 0$. Application at match $m(0) = v$: the fresh vertex w is connected to v by the single edge $\{v, w\}$ created in R. No other edges are created (the rule creates exactly one edge per match). No existing edges are modified.

Under GPI: multiple vertices matched simultaneously. Vertex v_i sprouts pendant w_i. The pendant w_i is adjacent to v_i only. Crucially, w_i is *not* adjacent to w_j for $i \neq j$ (the rule creates no edges between fresh vertices of distinct matches), and *not* adjacent to v_j for $j \neq i$ (the rule creates an edge only between the matched vertex and its pendant).

Therefore: $N_{G_t}(w_i) \cap V(G_{t-1}) = \{v_i\}$, giving $|N_{G_t}(w_i) \cap V(G_{t-1})| = 1$. □

Completion of Theorem 4.3. Edge subdivision of G_0 at any edge $\{u, v\} \in E(G_0)$ creates a fresh vertex w with $N(w) \supseteq \{u, v\}$, so $|N(w) \cap V(G_0)| \geq 2$. By Lemma 4.4, this cannot occur in any trajectory at level 2. Therefore $\mathrm{Sub}_1(G_0) \neq G_t$ for any t. □

Remark 4.5 (Generalization). Lemma 4.4 says: at level 2, every fresh vertex has *creation degree* 1 (pendant upon creation). At level 3, Rule 3.2 creates vertices with creation degree 2 (connected to both endpoints of the subdivided edge). Rule 3.3 also creates degree-2 vertices (connected to both endpoints of the matched edge). This structural observation—*creation degree increases with signature level*—suggests a general separation technique.

Conjecture 4.6 (Creation-Degree Separation)**.** *At level l (signature $l_2 \to (l{+}1)_2$), the maximum creation degree of a fresh vertex is* $\min(l, r{-}1)$*. At level $l+1$, fresh vertices with creation degree $l+1$ can be produced. The function "create a vertex of degree $l+1$ among previously existing vertices" separates level l from level $l+1$.*

4.3 Higher separations

For $l \geq k^*$: all levels are Turing-complete, so no separation exists.

For $3 \leq l < k^*$: each level is sub-universal but potentially capable of conditional branching ($l \geq 3$ provides multiple LHS patterns). The separation l vs. $l+1$ follows if we can exhibit a function computable at $l+1$ but not l. The Creation-Degree Conjecture would give this for all levels, but a proof requires formalizing the creation-degree bound as a function of l, which we leave open.

Between level $k^* - 1$ (sub-universal) and level k^* (Turing-complete): the separation is immediate—any Turing-computable function not in the sub-universal class witnesses it.

5 Universality

5.1 Overview

We construct a rule set at signature $7_2 \to 8_2$ that simulates an arbitrary Turing machine. The construction uses per-step priority between rules with *different LHS edge patterns* (but the same vertex count $l = 7$). Different edge patterns in the LHS encode different match conditions, giving the conditional branching needed for TM simulation. This is the mechanism that is unavailable at level 2 (where $L = K_2$ is the unique connected 2-vertex graph).

Why $l = 7$ and not less. The LHS must simultaneously contain:

- The current tape cell c_i (1 vertex),
- The pendant p_i of c_i, for symbol reading (1 vertex),
- The head marker, a K_3 subgraph encoding the TM state (3 vertices),
- Both neighboring cells c_{i-1} and c_{i+1}, for bidirectional movement (2 vertices).

Total: $1 + 1 + 3 + 2 = 7$ vertices.

5.2 Tape encoding

Cells. Each tape cell is a single vertex c_i. The tape is a path: c_0-c_1-c_2-$\cdots$-c_n. A left sentinel s is connected to c_0 (acting as c_{-1}). A right sentinel s_R is connected to c_n (acting as c_{n+1}).

Pendants. Every cell c_i has a pendant vertex p_i connected by edge $\{c_i, p_i\}$. The pendant is always present regardless of the tape symbol.

Symbol encoding. The symbol at cell c_i is encoded by a "leaning" edge:

- Symbol 1: edge $\{c_{i-1}, p_i\}$ exists (the pendant "leans left" toward the predecessor cell).
- Symbol 0: edge $\{c_{i-1}, p_i\}$ does not exist (the pendant connects only to c_i).

Head marker. A triangle K_3 on vertices $\{m_1, m_2, m_3\}$, attached to the current cell c_i by edge $\{c_i, m_1\}$. The marker is the unique triangle in the host graph (the tape path and pendants contain no triangles), ensuring exactly one match per step.

State encoding. TM state is encoded by the marker's internal edge pattern:

- State q_0 (active): all three edges $\{m_1, m_2\}$, $\{m_2, m_3\}$, $\{m_1, m_3\}$ present (K_3).
- State q_1 (halt): edge $\{m_2, m_3\}$ removed (marker is a path, not a triangle).

For TMs with more states, a larger marker (K_4 with $2^{\binom{4}{2}} = 64$ edge patterns) suffices. For the binary incrementer (2 states, 2 symbols), K_3 with 2 distinguishable patterns is enough.

5.3 The explicit DPO rules (binary incrementer)

The binary incrementer has transitions: $(q_0, 1) \to (q_0, 0, R)$ and $(q_0, 0) \to (q_1, 1, L)$.

LHS canonical labeling (7 vertices):

LHS vertex	Role	Notation
0	Left neighbor c_{i-1}	c_L
1	Current cell c_i	c
2	Pendant p_i	p
3	Marker vertex m_1	m_1
4	Marker vertex m_2	m_2
5	Marker vertex m_3	m_3
6	Right neighbor c_{i+1}	c_R

Table 2: LHS vertex assignment for the binary incrementer construction.

Base edges (present in both rules' LHS):

$$E_{\text{base}} = \big\{\{0,1\},\ \{1,2\},\ \{1,3\},\ \{1,6\},\ \{3,4\},\ \{4,5\},\ \{3,5\}\big\}.$$

Rule ρ_A: transition $(q_0, 1) \to (q_0, 0, R)$. Write 0, move right, stay q_0.

L_A: vertices $\{0, \ldots, 6\}$, edges $E_{\text{base}} \cup \{\{0, 2\}\}$. The edge $\{0, 2\} = \{c_L, p\}$ is the symbol-1 indicator. Total: 8 edges, connected.

R_A: vertices $\{0, \ldots, 7\}$ (vertex 7 = fresh), edges:

$$\big\{\{0,1\},\ \{1,2\},\ \{1,6\},\ \{3,4\},\ \{4,5\},\ \{3,5\},\ \{6,3\},\ \{1,7\}\big\}.$$

Interface map ι_A: identity on $\{0, \ldots, 6\}$.

Rule ρ_B: transition $(q_0, 0) \to (q_1, 1, L)$. Write 1, move left, enter halt state.

L_B: vertices $\{0, \ldots, 6\}$, edges E_{base} (no $\{0, 2\}$ edge—symbol 0). Total: 7 edges, connected.

R_B: vertices $\{0, \ldots, 7\}$ (vertex 7 = fresh), edges:

$$\big\{\{0,1\},\ \{1,2\},\ \{1,6\},\ \{3,4\},\ \{3,5\},\ \{0,2\},\ \{0,3\},\ \{0,7\}\big\}.$$

LHS edge	Status in R_A	Meaning
$\{0,1\}$	preserved	tape left
$\{1,2\}$	preserved	pendant stays
$\{1,3\}$	**deleted**	detach marker from c
$\{1,6\}$	preserved	tape right
$\{3,4\}$	preserved	marker K_3
$\{4,5\}$	preserved	marker K_3
$\{3,5\}$	preserved	marker K_3
$\{0,2\}$	**deleted**	write 0 (remove symbol indicator)
—	$\{6,3\}$ created	attach marker to c_R
—	$\{1,7\}$ created	fresh pendant on c (workspace)

Table 3: Edge changes for Rule ρ_A.

LHS edge	Status in R_B	Meaning
$\{0,1\}$	preserved	tape left
$\{1,2\}$	preserved	pendant stays
$\{1,3\}$	**deleted**	detach marker from c
$\{1,6\}$	preserved	tape right
$\{3,4\}$	preserved	marker edge
$\{4,5\}$	**deleted**	state $q_0 \to q_1$ (break K_3)
$\{3,5\}$	preserved	marker edge
—	$\{0,2\}$ created	write 1 (add symbol indicator)
—	$\{0,3\}$ created	attach marker to c_L (move left)
—	$\{0,7\}$ created	fresh pendant on c_L (workspace)

Table 4: Edge changes for Rule ρ_B.

Interface map ι_B: identity on $\{0,\ldots,6\}$.

Priority. $\rho_A > \rho_B$ (try ρ_A first at each step). Since L_A requires edge $\{0,2\}$ while L_B does not:

- Symbol 1 (edge $\{c_L, p\}$ present): ρ_A matches. ρ_A fires.
- Symbol 0 (edge $\{c_L, p\}$ absent): ρ_A fails. ρ_B matches. ρ_B fires.
- State q_1 (marker not K_3): neither matches (K_3 edges required by both). Halt.

5.4 Tape extension

Each TM step is simulated in one rewriting step for interior cells and at most two rewriting steps when the head reaches a tape boundary. This constant-factor slowdown does not affect the computability result. For the binary incrementer on a finite pre-allocated tape, boundary extension is not needed: the computation uses at most $n+1$ cells for an n-cell input.

5.5 Verified walkthrough: binary incrementer on $[1,1,0]$

Initial graph. Tape $[1,1,0]$ = binary 011 = 3. Head at c_0, state q_0.

Vertex	Role
0	Sentinel s $(= c_{-1})$
1	Cell c_0
2	Pendant p_0
3	Cell c_1
4	Pendant p_1
5	Cell c_2
6	Pendant p_2
7	Marker m_1
8	Marker m_2
9	Marker m_3
10	Right sentinel s_R $(= c_3)$

Table 5: Vertex assignments for the binary incrementer walkthrough.

Initial edges: Tape: $\{0,1\}$, $\{1,3\}$, $\{3,5\}$, $\{5,10\}$. Pendants: $\{1,2\}$, $\{3,4\}$, $\{5,6\}$. Symbol 1 at c_0: $\{0,2\}$ (s-to-p_0 leaning edge). Symbol 1 at c_1: $\{1,4\}$ (c_0-to-p_1 leaning edge). Symbol 0 at c_2: no leaning edge. Marker K_3: $\{7,8\}$, $\{8,9\}$, $\{7,9\}$. Head at c_0: $\{1,7\}$.

Step 1. Head at c_0. Match ρ_A: $0 \mapsto s = 0$, $1 \mapsto c_0 = 1$, $2 \mapsto p_0 = 2$, $3 \mapsto m_1 = 7$, $4 \mapsto m_2 = 8$, $5 \mapsto m_3 = 9$, $6 \mapsto c_1 = 3$. All required edges present. ρ_A **fires.** Delete $\{1,7\}$ (detach marker), delete $\{0,2\}$ (write 0). Create $\{3,7\}$ (attach marker to c_1), create $\{1,11\}$ (fresh pendant, vertex 11).

After step 1: Tape $[0,1,0]$. Head at c_1. State q_0.

Step 2. Head at c_1. Match ρ_A: need $\{0,2\} = \{1,4\}$. Symbol 1 at c_1 means $\{c_0, p_1\} = \{1,4\}$ exists. ρ_A **fires.** Delete $\{3,7\}$ (detach), delete $\{1,4\}$ (write 0). Create $\{5,7\}$ (attach to c_2), $\{3,12\}$ (fresh pendant, vertex 12).

After step 2: Tape $[0,0,0]$. Head at c_2. State q_0.

Step 3. Head at c_2. Check ρ_A: need $\{0,2\} = \{3,6\}$. Symbol 0 at c_2: edge $\{c_1, p_2\} = \{3,6\}$ does NOT exist. ρ_A **fails.** Try ρ_B: all base edges present. ρ_B **fires.** Delete $\{5,7\}$ (detach), delete $\{8,9\}$ (state $\to q_1$). Create $\{3,6\}$ (write 1), $\{3,7\}$ (attach marker to c_1), $\{3,13\}$ (fresh pendant, vertex 13).

After step 3: Tape $[0,0,1]$. Head at c_1. State q_1 (marker edges: $\{7,8\}$, $\{7,9\}$, missing $\{8,9\}$).

Step 4. Head at c_1. Check ρ_A: requires K_3 on marker, but edge $\{8,9\}$ absent. ρ_A **fails.** Check ρ_B: also requires K_3. ρ_B **fails.** No match. **HALT.**

Result: Tape $[0,0,1]$ = binary $100 = 4$. Computation $3 + 1 = 4$. ✓

5.6 General universality

Theorem 5.1 (Universality). *$k^* \leq 7$ for TMs with $q|\Gamma| \leq 8$; $k^* \leq 8$ for arbitrary TMs. Specifically: a rule set of $2q|\Gamma|$ rules at signature $7_2 \to 8_2$ (using a K_3 marker) simulates any TM with q states and alphabet Γ satisfying $q|\Gamma| \leq 8$. For larger TMs, use a K_4 marker ($2^6 = 64$ edge patterns), increasing the LHS to 8 vertices and the signature to $8_2 \to 9_2$. The construction uses:*

- *Cell encoding: 1 vertex per cell, 1 pendant per cell, symbol via leaning edge.*
- *Head marker: K_3 (or K_4 for > 8 state-symbol configurations).*

- *Both tape neighbors in LHS for bidirectional movement.*
- *Per-step priority between rules with different LHS edge patterns for conditional branching.*
- *Unique-match property: K_3 (resp. K_4) appears only in the marker.*

Any Turing machine can be simulated by a binary-alphabet TM with at most a polynomial overhead in time and a constant-factor overhead in tape [1]. The Neary–Woods $(15, 2)$ *UTM [10]—a standard-model universal Turing machine with 15 states and 2 symbols—gives* $q|\Gamma| = 30$*, which exceeds 8 and requires a K_4 marker (signature $8_2 \to 9_2$, giving $k^* \leq 8$). For TMs with $q|\Gamma| \leq 8$ (which suffice for any specific computation, though not for a single UTM), K_3 gives $k^* \leq 7$.*

The threshold k^* is identical for sequential and GPI application (Proposition 2.9(i)): the unique-match property ensures a single match per step.

Proof. The explicit construction for the binary incrementer (Section 5.3) demonstrates the mechanism. The walkthrough (Section 5.5) verifies correctness on a concrete input. The generalization to arbitrary TMs replaces the two rules ρ_A, ρ_B with a family of rules: one rule per (state, symbol) pair, each with the appropriate LHS edge pattern (encoding the symbol being read) and RHS edge pattern (encoding the symbol written, the direction moved, and the new state). Per-step priority orders these rules, with more specific LHS patterns (more edges) having higher priority. The construction is standard given the encoding; the key contribution is the encoding itself (leaning-edge symbol, K_3 marker, 7-vertex LHS window). □

6 Complexity bridge and logic connections

6.1 Signature complexity

Definition 6.1. $\sigma(f) = \min\{l : f \in \mathcal{C}_l^\alpha\}$ (or $\hat{\mathcal{C}}_l^\alpha$ under rule-set extension). This is well-defined for any computable graph function and equals k^* for Turing-complete functions.

6.2 Complexity bridge

Theorem 6.2 (Complexity bridge)**.** *For all $l \geq 1$, every rule ρ at signature $l_2 \to (l{+}1)_2$ can be simulated by a rule ρ' at signature $(l{+}1)_2 \to (l{+}2)_2$ such that:*

(a) *Function equivalence: $f_{(\rho',\alpha)}(G_0) = f_{(\rho,\alpha)}(G_0)$ for all G_0 in a cofinite subset of $\mathcal{G}$ (specifically, all graphs where every vertex in any match of L has degree ≥ 1).*

(b) *Description overhead: the rule ρ' can be specified from ρ using $C_l = O(l)$ additional bits.*

Proof. Construction. Given $\rho = (L, R, \iota)$, choose $v \in V(L)$. Define $L' = L + \{v'\}$ with edge $\{v, v'\}$ (so $|V(L')| = l + 1$, and L' is connected). Define $R' = R + \{v', w\}$ with edge $\{\iota(v), w\}$ (so $|V(R')| = l + 2$, and R' is connected if R is). The interface map: $\iota'(u) = \iota(u)$ for $u \in V(L)$; $\iota'(v') = v'$. Fresh vertex: w.

Function equivalence. A match of L' in G is an injective morphism mapping L to some subgraph of G AND mapping v' to a neighbor of $m(v)$ outside the match of L. This exists whenever $m(v)$ has a neighbor not in $m(V(L))$, which holds for all vertices of degree $\geq l$ in any

graph with $|V(G)| > l$. For such graphs, every match of L extends to a match of L', so the set of "active" positions is the same.

The local rewrite at each match position is: apply ρ (same effect on the L portion) plus create a pendant w attached to $\iota(v)$. The pendant w is workspace that does not interact with other parts of the graph. The computed function is the same (the additional pendant does not affect halting or the graph structure at fixed point, modulo the pendants—which can be stripped by convention).

Overhead. Specifying ρ' from ρ requires: choice of v ($\lceil \log_2 l \rceil$ bits), adjacency of v' in L' (l bits), adjacency of v', w in R' ($l+1$ bits). Total: $C_l = O(l)$. □

6.3 Relationship to Kolmogorov complexity

The interesting content of the complexity bridge is internal to the formalism: the signature ordering is monotone with description complexity. This is a statement about the specific GPI formalism as a programming language, not a Kolmogorov-complexity result (the invariance theorem gives the reverse inequality trivially for any formalism).

Corollary 6.3 (Signature monotonicity)**.** *The sequence of minimum description lengths $|S_l(f)|$ for a fixed function f is non-increasing in l: $|S_{l+1}(f)| \leq |S_l(f)| + C_l$. Combined with the strict separations (some functions require level $l+1$ and are not computable at level l), this gives a quantitative hierarchy: higher levels are strictly more expressive and every function at a lower level has a slightly longer description at the higher level.*

Proposition 6.4 (Description length of DPO rules)**.** *For any DPO rule ρ at signature $\sigma \to (\sigma{+}1)$ in* $\mathbf{Graph}_{\mathrm{inj}}$ *with connected L and R:*

$$K(\rho) = \Theta(\sigma^2).$$

Specifically: (i) $K(\rho) \leq \sigma^2 + O(\sigma \log \sigma)$, *and* (ii) $K(\rho) \geq \sigma^2/2 - O(\sigma)$ *for a positive fraction of all rules at level σ.*

Proof. (i) Encode $\rho = (L, R, \iota)$ canonically: $\binom{\sigma}{2}$ bits for the LHS adjacency, $\binom{\sigma+1}{2}$ bits for the RHS adjacency, $\sigma\lceil \log_2(\sigma+1) \rceil$ bits for the interface map, and $O(\log \sigma)$ bits for the header. Total: $\sigma^2 + O(\sigma \log \sigma)$ bits.

(ii) The number of distinct rules at level σ satisfies $N(\sigma) \geq c(\sigma) \cdot c(\sigma+1) \cdot (\sigma+1)!$, where $c(n) = 2^{\binom{n}{2}} \cdot (1 - o(1))$ is the number of connected labeled graphs on n vertices (Erdős–Rényi 1959). So $N(\sigma) \geq 2^{\sigma^2 + O(\sigma \log \sigma)}$. By a counting argument, at most $2^{B+1} - 1$ objects can have Kolmogorov complexity $\leq B$. Setting $B = \sigma^2/2 - \sigma$, we get $2^{B+1} \ll N(\sigma)$, so most rules have $K(\rho) > \sigma^2/2 - \sigma$. □

Remark 6.5. Within-level variation in $K(\rho)$ is $O(\sigma \log \sigma)$—sub-leading compared to the σ^2 inter-level gap. This justifies stating the PRIMO conjecture [8] at the level of σ rather than individual $K(\rho)$: the within-level variation is noise relative to the between-level signal.

6.4 Connections to bounded-variable logics

Static correspondence. The LHS size l corresponds to the pebble count k in Immerman's k-pebble game [7]. A match of L in G is an injection from l vertices to G—a "pebble placement." Two graphs are $\mathcal{L}^l_{\infty\omega}$-equivalent (Duplicator wins the l-pebble game) iff they agree on

all subgraph counts for patterns of size $\leq l$. Since the match count $|\{m : L \hookrightarrow G\}|$ for $|V(L)| = l$ is determined by $\mathcal{L}^l_{\infty\omega}$-equivalence class (Lovász 1967 [9]; Dell–Grohe–Rattan 2018 [5]):

Proposition 6.6. *If $G \equiv_{\mathcal{L}^l_{\infty\omega}} H$, then for any rule ρ at level l, the number of matches of L in G equals the number of matches of L in H.*

This means the l-pebble equivalence is a sufficient condition for two graphs to "look the same" to any level-l rule at a single step.

Dynamic breakdown. Over multiple steps, the correspondence fails: a level-l rule applied for t steps effectively "scans" $O(t \cdot l)$ vertices, accumulating information in the graph's evolving structure. The system's discriminating power after t steps exceeds $\mathcal{L}^l_{\infty\omega}$. A precise characterization would require a *temporal bounded-variable logic*—something like l-variable logic with a constructive least-fixed-point operator, restricted to operations realizable by DPO rules.

Courcelle's MSO transductions. Checking whether a fixed pattern L has an injective match in G is first-order definable (a sentence with l existentially quantified vertex variables). Selecting a maximal independent match set is MSO-definable. The iterated GPI application of a fixed rule is therefore an iterated MSO-definable transduction. Courcelle's theory [3, 4] provides general tools (closure under composition, decidability on bounded-treewidth graphs) that may yield upper bounds on our formalism's power. Specifically: if a class of graphs has bounded treewidth, then all MSO-definable properties of its elements are decidable. Our lower-level rules produce graphs of bounded treewidth (level-2 sprouting produces trees), so Courcelle's theorem applies and gives decidability of trajectory properties—consistent with the "finite-state-like" behavior at low levels.

7 Computational illustration

See Section 5.5 for the binary incrementer walkthrough. The companion study [8] evaluates Rule 3.2 (subdivision) on initial graphs K_1, K_2, K_3, P_3 and classifies it as I^+ and Φ^+. The theoretical hierarchy places this rule at the first level with topological context ($l = 2$), consistent with the empirical observation that complex dynamical behavior requires context-sensitivity (the L-system analogy: context-free D0L produces regular behavior; context-dependent IL produces complex behavior).

Rule count reconciliation. This paper catalogs 5 connected rules across 3 signature levels. The companion study's ∼8,000 count arises from: (i) signatures $l_2 \to r_2$ with $r \neq l + 1$ (including $2_2 \to 2_2$, $1_2 \to 3_2$, $3_2 \to 4_2$, etc.); (ii) disconnected R; (iii) rule instances on specific initial graphs from $\mathcal{G}_0 = \{K_1, K_2, K_3, P_3\}$ (multiplying by 4); (iv) higher signatures. The theoretical hierarchy depends only on LHS size and the $r = l + 1$ convention.

8 Open problems

1. **Conditional branching threshold.** At what LHS size does conditional branching first become possible? At $l = 2$, all rules share $L = K_2$ and no conditional behavior is achievable (Appendix C). At $l = 7$, the universality construction implements conditional branching via different LHS edge patterns. What happens at $l = 3, 4, 5, 6$? At $l = 3$, there are two connected LHS patterns (K_3 and P_3), so rules can have genuinely different match conditions. Can conditional branching at $l = 3$ simulate counter machines? DPDAs?

2. **Tighten k^*.** Current: $k^* \leq 7$ (with K_3 marker, for TMs with $q|\Gamma| \leq 8$) or $k^* \leq 8$ (with K_4 marker, for arbitrary TMs via the Neary–Woods UTM). Can the LHS be reduced to 6 vertices? A lower bound on k^* would require proving that no rule set at signature $6_2 \to 7_2$ can simulate an arbitrary TM.

3. **Sequential vs. GPI.** At sub-universal levels, is GPI strictly more powerful? Characterize when trajectories coincide (confluence conditions).

4. **Justify $r = l + 1$ formally.** Prove that for all l and $j \geq 2$, $\mathcal{C}^\alpha(l, l+j) = \mathcal{C}^\alpha(l, l+1)$ (multi-step iteration at $r = l + 1$ covers the same class as single-step at $r = l + j$).

5. **Descriptive complexity.** Find logic $\mathcal{L}_l$ matching level-l GPI power.

6. **Creation-degree separation.** Formalize the conjecture that level l cannot create vertices of degree $> l - 1$ among existing vertices, and prove it for all l.

7. **Ordering independence.** Prove or disprove: the computational class $\mathcal{C}_l^{\mathrm{GPI}}$ is independent of the vertex ordering on the host graph.

8. **Hyperedge generalization.** Extend to arity-a hyperedges. How does (l, r, a) affect the hierarchy?

A Notation

Symbol	Meaning
$l_2 \to r_2$	Signature
$\rho = (L, R, \iota)$	Rule with interface map
$\mathcal{C}_l^\alpha$	Function class (halting-based)
$\mathcal{D}_l^\alpha$	Dynamics class (trajectory-based)
$\hat{\mathcal{C}}_l^\alpha, \hat{\mathcal{D}}_l^\alpha$	Extended classes (rule sets, per-step priority)
$f_\mathcal{S}$	Graph function of system $\mathcal{S}$
$\tau_\mathcal{S}$	Trajectory map of system $\mathcal{S}$
k^*	Universality threshold (≤ 7 or ≤ 8)

Table 6: Summary of notation.

B Complete rule catalog

See the repository file `rule_catalog.md` for the exhaustive enumeration and verification of all rules at signatures $1_2 \to 1_2$, $1_2 \to 2_2$, and $2_2 \to 3_2$.

C DPDA impossibility at $2_2 \to 3_2$

The formal impossibility is proved in Theorem 3.7. We summarize the key points here for context.

The obstruction has three layers:

1. **Shared LHS (Lemma 3.5).** All connected rules at $2_2 \to 3_2$ have $L = K_2$. Per-step priority between rules with identical LHS is vacuous: the highest-priority rule always fires. The rule-set extension adds no power.

2. **Operation uniformity.** A single rule applies the same fixed operation (sprout, subdivide, or triangulate) at every matched edge. The system cannot apply different operations to different edges, or different operations at different steps.

3. **DPDA requires branching.** A DPDA transition requires inspecting the stack top and choosing between push, pop, or replace. This $|\Gamma|$-way branching cannot be encoded through matching-cardinality differences of a single fixed rule.

Could alternative mechanisms help?

- **Per-match priority** (choosing between rules at each match based on local structure): vacuous, since all rules share $L = K_2$—any match of one rule is a match of all rules.
- **Negative application conditions (NACs):** a NAC could make rule firing context-dependent. NACs are not part of our plain DPO model.
- **Vertex labels or colors:** equivalent to having different LHS patterns. Not part of our model.

The sharp boundary is at level 4 ($l = 3$), where two connected LHS patterns (K_3 and P_3) enable conditional branching for the first time.

Acknowledgments

This work is part of the PRIMO research programme. Code and data are available at https://github.com/KarolFilipKowalczyk/PRIMO.

References

[1] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[2] Bruno Courcelle. The monadic second-order logic of graphs I: Recognizable sets of finite graphs. *Information and Computation*, 85:12–75, 1990.

[3] Bruno Courcelle. Monadic second-order definable graph transductions: A survey. *Theoretical Computer Science*, 126:53–75, 1994.

[4] Bruno Courcelle and Joost Engelfriet. *Graph Structure and Monadic Second-Order Logic: A Language-Theoretic Approach*. Cambridge University Press, 2012.

[5] Holger Dell, Martin Grohe, and Gaurav Rattan. Lovász meets Weisfeiler and Leman. In *Proceedings of the 45th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2018.

[6] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. EATCS Monographs in Theoretical Computer Science. Springer, 2006.

[7] Neil Immerman. Upper and lower bounds for first order expressibility. *Journal of Computer and System Sciences*, 25:76–98, 1982.

[8] Karol Filip Kowalczyk. Geometric predicates for classifying dynamical behaviors in graph rewrite systems. In preparation, 2026.

[9] László Lovász. Operations with structures. *Acta Mathematica Academiae Scientiarum Hungaricae*, 18:321–328, 1967.

[10] Turlough Neary and Damien Woods. Four small universal Turing machines. *Fundamenta Informaticae*, 91(1):123–144, 2009.

[11] Grzegorz Rozenberg and Arto Salomaa. *The Mathematical Theory of L Systems*. Academic Press, 1980.